# DDA3020 Machine Learning

Tutorial: Convolutional Neural Networks

April 2026

## Instructions

This exercise sheet covers the main ideas of convolutional neural networks (CNNs), including the basic CNN architecture, convolution layers, activation maps, spatial dimensions, stride, zero padding, pooling layers, and fully connected layers. You should understand how convolution filters operate on an input volume, how to compute the output size of a convolution layer, and how to count the number of learnable parameters. You should also be familiar with the roles of pooling and fully connected layers in a CNN.

## Useful Formulas

**1. Output spatial size without padding**

For a convolution layer with input size $N$, filter size $F$, and stride $S$,

$$\text{output size} = \frac{N - F}{S} + 1.$$

This must be an integer for the configuration to be valid.

**2. Output spatial size with padding**

For a convolution layer with input size $N$, filter size $F$, stride $S$, and padding $P$,

$$\text{output size} = \frac{N + 2P - F}{S} + 1.$$

**3. Output volume size**

If the input volume is

$$W_1 \times H_1 \times C$$

and the convolution layer uses $K$ filters of size $F \times F \times C$, stride $S$, and padding $P$, then the output volume is

$$W_2 \times H_2 \times K,$$

where

$$W_2 = \frac{W_1 - F + 2P}{S} + 1, \qquad H_2 = \frac{H_1 - F + 2P}{S} + 1.$$

**4. Number of parameters in a convolution layer**

If a convolution layer uses $K$ filters of size $F \times F \times C$, then:

$$\text{parameters per filter} = F^2C + 1,$$

where the extra 1 is the bias term.

So the total number of learnable parameters is

$$K(F^2C + 1).$$

Equivalently,

$$F^2CK + K.$$

**5. Padding for preserving spatial size**

When stride $S = 1$ and the filter size $F$ is odd, a common choice of padding to preserve spatial size is

$$P = \frac{F - 1}{2}.$$

Examples:

$$F = 3 \Rightarrow P = 1, \qquad F = 5 \Rightarrow P = 2, \qquad F = 7 \Rightarrow P = 3.$$

**6. Interpretation reminders**

- One filter produces one activation map.
- If a layer has $K$ filters, then the output depth is $K$.

- A pooling layer reduces spatial dimensions and operates independently on each activation map.
- A fully connected layer connects each neuron to the entire input volume.

# Lecture Exercises

**Q1.** [Source: pp. 38–48] Consider a convolution operation on an input of spatial size $7 \times 7$ using a $3 \times 3$ filter and no padding.

1. Find the output spatial size when the stride is 1.
2. Find the output spatial size when the stride is 2.
3. Determine whether stride 3 is valid.

**Q2.** [Source: pp. 49–51] Suppose the input has spatial size $7 \times 7$. A $3 \times 3$ filter is applied with stride 1, and the input is padded with a 1-pixel border. Find the output spatial size.

**Q3.** [Source: pp. 23–25, 35–37, 52–53] Consider an RGB input image of size $32 \times 32 \times 3$. A convolution layer applies 10 filters of size $5 \times 5 \times 3$, with stride 1 and padding 2.

1. Find the spatial size of the output.
2. Find the full output volume size.
3. Explain how the number of filters affects the depth of the output volume.

**Q4.** [Source: pp. 23–24] Suppose an input volume of size $32 \times 32 \times 3$ passes through two convolution layers in sequence:

- the first layer uses 6 filters of size $5 \times 5 \times 3$,
- the second layer uses 10 filters of size $5 \times 5 \times 6$.

Assume stride 1 and no padding for both layers. Find the output volume after each layer.

**Q5.** [Source: pp. 54–55] Consider a convolution layer with input volume size $32 \times 32 \times 3$. The layer uses 10 filters of size $5 \times 5 \times 3$. Find the total number of learnable parameters.

# Additional Exercises

**Q1.** Which of the following is an example of a many-to-one task?

(A) Image classification

(B) Image captioning

(C) Text classification

(D) Machine translation

**Q2.** Why are recurrent neural networks more suitable than ordinary feed-forward neural networks for sequential data?

**Q3.** Consider the RNN update

$$h_t = f_W(h_{t-1}, x_t).$$

Explain the meaning of each term in this expression.

**Q4.** Which statement is correct about an RNN?

(A) It uses different parameters at every time step

(B) It shares the same parameters across time steps

(C) It ignores the previous hidden state

(D) It cannot process variable-length sequences

**Q5.** The lecture defines the cost function

$$E(\theta) = \frac{1}{T} \sum_{t=1}^{T} L(y_t, \hat{y}_t).$$

Explain the meaning of $y_t$, $\hat{y}_t$, $L(\cdot, \cdot)$, and $T$.

**Q6.** Why is backpropagation through time needed for training RNNs?

(A) Because RNNs only have one layer

(B) Because the hidden state depends on previous time steps, so gradients must be propagated through the sequence

(C) Because RNNs do not use gradient descent

(D) Because the output does not depend on the hidden state

**Q7.** What is the idea of truncated backpropagation through time?

**Q8.** Which of the following best describes gradient exploding?

(A) Gradients become exactly zero at every time step

(B) Gradients become extremely small during training

(C) Gradients become exceptionally large, causing numerical instability

(D) The model stops updating its parameters completely

**Q9.** Why does gradient vanishing make learning long-range dependencies difficult?

(A) Because the gradients become too small as they are backpropagated through time

(B) Because the hidden state is always zero

(C) Because the model cannot compute outputs

(D) Because the vocabulary size is too large

**Q10.** What is gradient clipping, and how can it help during training?

**Q11.** What problem was LSTM designed to address?

**Q12.** Why is LSTM generally better than a basic RNN for modeling long-distance dependencies?

(A) Because it removes the need for hidden states

(B) Because it provides an easier way for the model to learn long-distance dependencies

(C) Because it uses no parameters

(D) Because it always eliminates all gradient problems

**Q13.** Which statement about LSTM is correct according to the lecture?

(A) LSTM guarantees that gradients will never vanish or explode

(B) LSTM is identical to a basic RNN

(C) LSTM helps with long-distance dependencies but does not completely guarantee the absence of vanishing or exploding gradients

(D) LSTM cannot be trained with gradient-based methods

**Q14.** What is a gated recurrent unit (GRU)?

**Q15.** According to the lecture, what is one important structural difference between GRU and LSTM?

(A) GRU has more gates than LSTM

(B) GRU has fewer parameters than LSTM because it lacks an output gate

(C) GRU does not use recurrent connections

(D) GRU cannot process sequential data

**Q16.** State two major limitations of basic RNNs and their variants discussed in the lecture.

**Q17.** Why is parallelization difficult for RNNs?

(A) Because RNNs process data sequentially across time steps

(B) Because RNNs do not use matrix multiplication

(C) Because RNNs have no hidden states

(D) Because RNNs cannot be trained on GPUs

**Q18.** What is a transformer, and what mechanism is it based on?

**Q19.** Why are positional encodings needed in a transformer?

(A) Because transformers process tokens in parallel and need extra information to represent order

(B) Because positional encodings replace input embeddings

(C) Because positional encodings reduce the vocabulary size

(D) Because positional encodings are only used in CNNs

**Q20.** In the lecture, each encoder is described as

$$\text{Self-Attention} \rightarrow \text{Add \& Norm} \rightarrow \text{MLP} \rightarrow \text{Add \& Norm}.$$

Briefly explain the role of this encoder structure.

**Q21.** Compare RNNs and transformers in terms of:

1. parallelization,
2. handling long-range dependencies,
3. model capacity and flexibility.

# Solutions to Lecture Exercises

**Q1. Answer:** For convolution without padding, the output spatial size is

$$\frac{N-F}{S}+1.$$

Here $N = 7$ and $F = 3$.

**Explanation:**

- For stride 1:
$$\frac{7-3}{1}+1=5.$$
So the output size is $5 \times 5$.
- For stride 2:
$$\frac{7-3}{2}+1=3.$$
So the output size is $3 \times 3$.
- For stride 3:
$$\frac{7-3}{3}+1=\frac{4}{3}+1=2.33\ldots$$
Since the output size must be an integer, this setting is invalid.

**Q2. Answer:** The output size is $7 \times 7$.

**Explanation:** With padding, the output spatial size is

$$\frac{N+2P-F}{S}+1.$$

Substituting $N = 7$, $P = 1$, $F = 3$, and $S = 1$ gives

$$\frac{7 + 2(1) - 3}{1} + 1 = 7.$$

So the spatial size is preserved.

**Q3. Answer:** The output spatial size is $32 \times 32$, the full output volume is $32 \times 32 \times 10$, and the number of filters determines the output depth.

**Explanation:** Using

$$\frac{N + 2P - F}{S} + 1,$$

with $N = 32$, $P = 2$, $F = 5$, and $S = 1$, we get

$$\frac{32 + 4 - 5}{1} + 1 = 32.$$

So the spatial size is $32 \times 32$. Since each filter produces one activation map and there are 10 filters, the output depth is 10.

**Q4. Answer:** The output volumes are

$$32 \times 32 \times 3 \to 28 \times 28 \times 6 \to 24 \times 24 \times 10.$$

**Explanation:** For the first layer,

$$\frac{32 - 5}{1} + 1 = 28,$$

so the output is $28 \times 28 \times 6$. For the second layer,

$$\frac{28 - 5}{1} + 1 = 24,$$

so the output is $24 \times 24 \times 10$.

**Q5. Answer:** The total number of learnable parameters is 760.

**Explanation:** Each filter has

$$5 \times 5 \times 3 = 75$$

weights and one bias term, so each filter has 76 parameters. Since there are 10 filters, the total is

$$76 \times 10 = 760.$$

# Solutions to Additional Exercises

**Q1. Answer: (C)** Text classification.

**Explanation:** A many-to-one task takes a sequence as input and outputs a single label or prediction.

**Q2. Answer:** RNNs are more suitable because they maintain a recurrent hidden state, share parameters across time, and can model order and dependencies in variable-length sequences.

**Explanation:** Unlike ordinary feed-forward neural networks, RNNs explicitly model temporal dependence by passing information from one time step to the next.

**Q3. Answer:** In

$$h_t = f_W(h_{t-1}, x_t),$$

$h_t$ is the hidden state at time step $t$, $h_{t-1}$ is the previous hidden state, $x_t$ is the current input, and $f_W$ is the transformation function parameterized by $W$.

**Q4. Answer: (B)** It shares the same parameters across time steps.

**Explanation:** Parameter sharing across time is one of the defining characteristics of RNNs.

**Q5. Answer:** $y_t$ is the ground-truth target at time step $t$, $\hat{y}_t$ is the model prediction at time step $t$, $L(y_t, \hat{y}_t)$ is the loss at time step $t$, and $T$ is the total number of time steps.

**Explanation:** The cost function averages the loss over the entire sequence.

**Q6. Answer: (B)** Because the hidden state depends on previous time steps, so gradients must be propagated through the sequence.

**Explanation:** Training an RNN requires propagating gradients through the unrolled computational graph over time.

**Q7. Answer:** Truncated backpropagation through time carries hidden states forward through the sequence but only backpropagates gradients for a limited number of time steps.

**Explanation:** This reduces computational cost and memory usage compared with full backpropagation through time.

**Q8. Answer: (C)** Gradients become exceptionally large, causing numerical instability.

**Explanation:** Exploding gradients can make optimization unstable and cause numerical overflow.

**Q9. Answer: (A)** Because the gradients become too small as they are backpropagated through time.

**Explanation:** Very small gradients mean early time steps receive little useful training signal, making long-range dependency learning difficult.

**Q10. Answer:** Gradient clipping restricts gradients to a specified range or norm threshold to prevent them from becoming excessively large.

**Explanation:** It helps stabilize training and reduce numerical instability caused by exploding gradients.

**Q11. Answer:** LSTM was designed to address the vanishing gradient problem and improve the modeling of long-distance dependencies.

**Explanation:** Its gating structure provides an easier path for information to persist across many time steps.

**Q12. Answer: (B)** Because it provides an easier way for the model to learn long-distance dependencies.

**Explanation:** LSTM improves over a basic RNN by making long-range information flow easier to learn.

**Q13. Answer: (C)** LSTM helps with long-distance dependencies but does not completely guarantee the absence of vanishing or exploding gradients.

**Explanation:** The lecture explicitly notes that LSTM does not fully eliminate these problems.

**Q14. Answer:** A GRU is a type of recurrent neural network that uses gating mechanisms to control the flow of information through time.

**Explanation:** It is a simplified gated recurrent model compared with LSTM.

**Q15. Answer: (B)** GRU has fewer parameters than LSTM because it lacks an output gate.

**Explanation:** According to the lecture, this is a key structural difference between the two models.

**Q16. Answer:** Two major limitations are difficulty with long-term dependencies and limited parallelization.

**Explanation:** RNNs struggle with vanishing gradients and must process sequences step by step.

**Q17. Answer: (A)** Because RNNs process data sequentially across time steps.

**Explanation:** Each time step depends on the previous hidden state, which makes parallel computation difficult.

**Q18. Answer:** A transformer is a deep learning architecture based on the attention mechanism, especially multi-head attention.

**Explanation:** The lecture introduces transformers as attention-based models with input embeddings and positional encodings.

**Q19. Answer: (A)** Because transformers process tokens in parallel and need extra information to represent order.

**Explanation:** Without positional encodings, the model would not know token order.

**Q20. Answer:** This encoder structure first lets tokens interact through self-attention, then applies residual connection and normalization, followed by an MLP and another residual normalization step.

**Explanation:** It combines contextual interaction and feed-forward transformation while improving training stability.

**Q21. Answer:** Compared with RNNs, transformers are easier to parallelize, more effective at handling long-range dependencies, and usually have higher model capacity and flexibility.

**Explanation:** RNNs are sequential and often struggle with long-term dependencies, while transformers use attention to model global relationships more efficiently.